

ICPC Bootcamp Day 1 Summary

Based on `day1_slides.pptx`

Session Summary

Day 1 builds the basic vocabulary for competitive programming: asymptotic analysis, abstract data types, and the data structures that appear repeatedly in later algorithms. The session starts with time and space complexity. Big-O notation is used to describe how runtime or memory grows as input size grows, and students are reminded that complexity estimates are practical contest tools. They help decide whether an approach can pass before the team spends time implementing it.

The deck then introduces abstract data types: sets, lists, maps, graphs, queues, priority queues, stacks, and deques. The purpose is to separate the idea of a structure from a particular implementation. A map promises key-value lookup; in Python it is normally implemented with a dictionary, while other languages may use hash tables or balanced trees. A priority queue promises access to the next highest-priority item; in Python this is normally a binary heap through `heapq`. This distinction helps students choose structures based on required operations rather than names alone.

Several built-in and library-supported structures are covered in detail. Arrays and dynamic arrays provide indexed storage and efficient iteration, with sorting and binary search as common operations. Linked lists are introduced mainly to explain stacks, queues, and deques, while also warning that linked lists are often not the practical choice for contest list operations. Stack-based reasoning is applied to bracket matching: push opening brackets, pop matching closing brackets, and accept only if the stack is empty at the end.

The non-linear data structure section focuses on binary heaps, hash tables, and balanced binary search trees. Binary heaps implement priority queues and support efficient insertion and extraction of the minimum or maximum. Hash tables support expected constant-time lookup, insertion, deletion, membership tests, and frequency counts, but students should remember that collision behavior explains the worst case. Balanced search trees keep keys ordered, making them useful when both search and sorted traversal are needed.

The graph representation portion introduces adjacency matrices, adjacency lists, and edge lists. Adjacency matrices are simple and good for dense or small graphs but use $\mathcal{O}(V^2)$ space. Adjacency lists are the default for most sparse graph algorithms because they store only existing edges and enumerate neighbors efficiently. Edge lists are useful when algorithms need to process all edges globally, especially Kruskal's minimum spanning tree algorithm.

The final major topics are bitmasks, union-find disjoint sets, and Fenwick trees. Bitmasks represent small sets as integers and use constant-time bit operations for membership, insertion, deletion, toggling, full-set construction, and least-significant-bit extraction. Union-find models changing connected components using parent arrays, representatives, path compression, union by rank, set counts, and set sizes. Fenwick trees accelerate range sum queries and point updates by storing cumulative chunks based on least significant bits, improving over naive prefix-sum updates when the array changes.

Key Takeaways

- Analyze constraints before choosing an algorithm or data structure.
- Choose data structures by operation needs: lookup, ordered traversal, neighbor enumeration, priority extraction, or dynamic range sums.
- Adjacency lists are the default graph representation; edge lists are natural for Kruskal; matrices fit small dense graphs.
- Bitmasks, union-find, and Fenwick trees are compact contest tools that turn many slow operations into efficient ones.

CP4 Reading Connections

- CP4 Section 1.3.3, *Do Algorithm Analysis*.
- CP4 Section 2.2, *Linear DS with Built-in Libraries*, especially Sections 2.2.1, 2.2.3, 2.2.5, and 2.2.6.
- CP4 Section 2.3, *Non-Linear DS with Built-in Libraries*, especially Sections 2.3.1, 2.3.2, and 2.3.3.
- CP4 Section 2.4, *DS with Our Own Libraries*, especially Sections 2.4.1, 2.4.2, and 2.4.3.